Clean software development

Day 4

David A. Clarke

Z02 – Software Development Center

9 October 2025

GOETHE UNIVERSITÄT FRANKFURT AM MAIN

clarkedavida

CRC-TR 211
Strong-interaction matter under extreme conditions

HGS-HIRe for FAIR
Helmholtz Graduate School for Hadron and Ion Research

Topics of the day

1. Motivation
2. Validating data
3. Cleaning data
4. 2D Ising model (I)
5. Plotting data
6. Estimating uncertainties
7. 2D Ising model (II)

Cross checking

Today the goal is to learn and practice several strategies to validate and clean data.

We will write our own code from scratch!

Motivation

Objectives and Expectations

We are going to **learn good practices curating scientific data**
through a small data analysis project.

Everything I present here I learned by watching excellent scientists.
This is my attempt to emulate them.

I try to keep lecture roughly self-contained.

Show examples from my own work, e.g. the 🔗AnalysisToolbox .

Please interrupt at any time!

Testing and validation

- Testing: Carrying out **checks of functionality and correctness** of code
- Validation: Testing that focuses in particular on the **scientific correctness** of code
- As always, driven by **falsification principle**

My personal tower of priorities:

1. Code output is **consistent**
   - Self-consistency
   - Consistent with what we "know"
2. Code is **clean** and **understandable**
3. Code is **efficient**

Everything else (elegant, clever, cute, etc.) is fluff.

Validating data

Validation strategies

Here are some ideas to check consistency of code output.
I think of them as Cross Checks (CC).
Some are the **same checks you do in physics**.

What I use most commonly:

1. Compare against trusted result
2. Compute a result multiple ways
3. Check for consistency between outputs
4. Check known properties
5. Verify independence of hyperparameters

The list above is not meant to have a particular order, is not exhaustive, and is not disjoint!

CC1: Compare against trusted result

Often a starting point: 🔗 reproduce something

Particularly nice example:

Bayesian model averaging gives a way to systematically incorporate systematic uncertainty from model choice (e.g. choice of fit function, number of discarded data) into total uncertainty. Linked paper above gives a mock example that you can directly reproduce.

C. Practical example: polynomial data

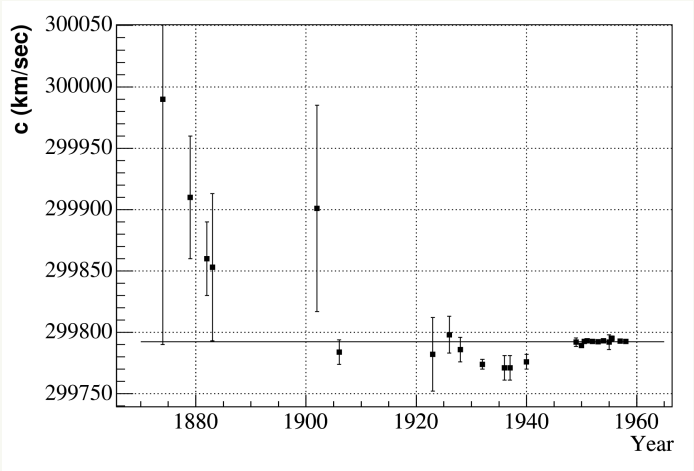
To demonstrate the method and some key features, we begin by considering a simple toy model. We begin by specifying a quadratic "model truth" polynomial function,

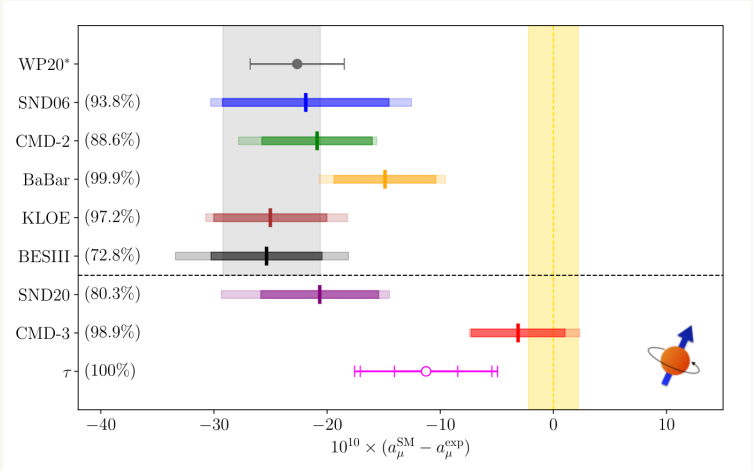
$$F(x) = 1.80 - 0.53\left(\frac{x}{16}\right) + 0.31\left(\frac{x}{16}\right)^2. \qquad (39)$$

CC1: Compare against trusted result

Danger with reproducing: you **assume other calculation is correct**

- Has the result already been reproduced?
- You could apply some Cross Checks to the trusted result
- 🔗Blinding can help against human bias:
  1. Let $r \in [-R, R]$ unknown to you
  2. Rescale your observable by r
  3. Remove r when you are confident in your calculation





CC1: Compare against trusted result

Other sources of "trusted" results:

- Can test against your old results to **ensure stability**
- Construct simple case you can **work out by hand**
  - Set pieces of the test case to 0 or 1
  - Reduce the size of the test case
- If you already went through the trouble of doing one of these comparisons, **you may as well add it to your tests**

Examples

- LQCD: work out result for small lattice with all matrices set to 1
- Statistics: Work out an example with 5 measurements only.

CC2: Compute a result multiple ways

There's usually more than one way to solve a problem

- Check optimized implementation against easy-to-understand one
- Test different algorithms against each other
- Test different models against each other

«Omnes viae Romam ducunt»
↳ All roads lead to Rome
Augustus

CC2: Compute a result multiple ways

Ex: Pure Python vs Numpy

$$\mathrm{acint}(T) = 1 + 2\sum_{t=1}^{T-1} \frac{\mathrm{autocor}(t)}{\mathrm{autocor}(0)}$$

```
# Want 2-d array. One axis for jackknife bin. One axis for T.
acintj = [[1.]*Nbins]
for t in range(1,Nt+1):
    tauintbins = []
    for bin in range(Nbins):
        tauintbins.append(
            acintj[t-1][bin] + 2*acorj[t][bin]/acorj[0][bin]
        )
    acintj.append(tauintbins)


# Faster numpy implementation. Compare acintj from both methods!
acintj      = np.ones((Nt+1,Nbins))
acintj[1:] = np.cumsum(2*acorj[1:]/acorj[0],axis=0) + 1
```


CC3: Check for consistency between outputs

Observables may be constrained by relationships between them. Design tests that ensure observables obey these known constraints.

Examples?

CC3: Check for consistency between outputs

Observables may be constrained by relationships between them. Design tests that ensure observables obey these known constraints.

Examples

- Speed, velocity, acceleration
- n_B and p as function of μ_B
- **Gibbs-Duhem:** $Ts = \epsilon + p - \sum_i \mu_i n_i$
- $f\left(f^{-1}\left(x\right)\right) = x$

CC3: Check for consistency between outputs

Ex: Round-trip test

Sometimes you will transform your data, for example when switching between different formats. A **round-trip** test checks that data remain consistent and accurate through these transformations.

```
# Make reference table
ref = latexTable()
ref.append(['1','2','3'])
ref.append(['1','2','3'])
ref.writeTable('test.tex')

# Round-trip test
convertTable('test.tex','test.csv',targetDelimiter=',')
convertTable('test.csv','test.tex',sourceDelimiter=',')
test.readTable('testInterface.tex')
lpass *= test==ref
```


CC4: Check known properties

Depending on an observable's interpretation, you may already know some properties it must have. Test them!

Examples?

CC4: Check known properties

Depending on an observable's interpretation, you may already know some properties it must have. Test them!

Examples

- is your covariance matrix positive semidefinite?
- is your unitary matrix actually unitary?
- is your RHMC reversible?
- is your probability between 0 and 1?
- are physical observables at physical parameters real?

CC4: Check known properties

Ex: A check for SU(N) class methods

```
class SUN(np.ndarray):
    ...
    def isSUN(self) -> bool:
        """ Check that I have det=1 and am unitary. """
        if not (isSpecial(self) and isUnitary(self)):
            return False
        return True

class SU3(SUN):
    ...

g = SU3()
g.setToRandom()

lpass *= g.isSUN()
```

CC5: Verify independence of hyperparameters

- Parameter: Directly **model-relevant** number
  - Temperature, volume, particle mass, coupling constant
  - Fit parameters
- Hyperparameter: Needed **only for computational strategy**
  - Jackknife bins, bootstrap samples
  - Initial guess of a fitter
  - Max iterations, solve tolerance, CNN kernel size
- Often only independent of hyperparameter **in some region**
  - Number of jackknife bins until error bar plateaus
  - Initial guesses leading to reasonable $\chi^2/\mathrm{d.o.f.}$

CC5: Verify independence of hyperparameters

Ex: Convergence of fitter

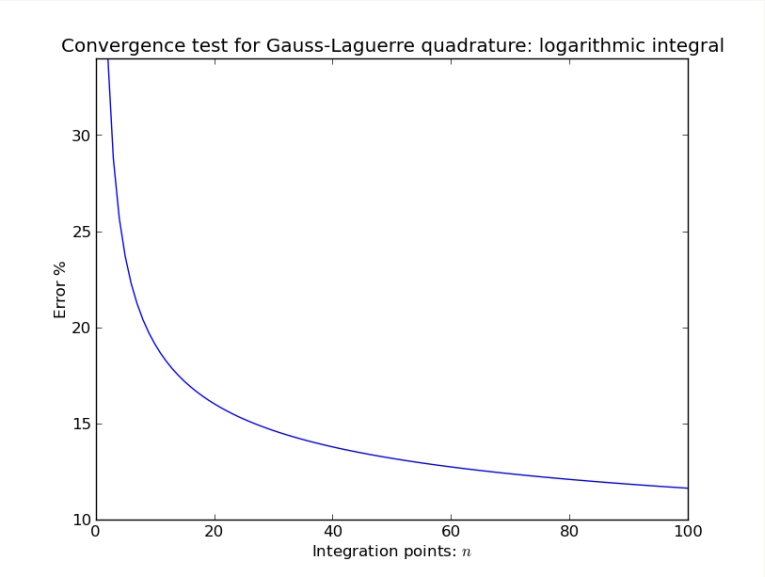
```
#          tol              res[0]       chi^2/d.o.f.
     1.0e+00   -1.34121869e+00    5.76571040e-01
     5.0e-01   -1.94341345e+00    2.59652760e-01
     1.0e-01   -1.94341345e+00    2.59652760e-01
     1.0e-02   -1.94341345e+00    2.59652760e-01
```


CC5: Verify independence of hyperparameters

Ex: Convergence of fitter

# tol	res[0]	chi^2/d.o.f.
1.0e+00	-1.34121869e+00	5.76571040e-01
5.0e-01	-1.94341345e+00	2.59652760e-01
1.0e-01	-1.94341345e+00	2.59652760e-01
1.0e-02	-1.94341345e+00	2.59652760e-01

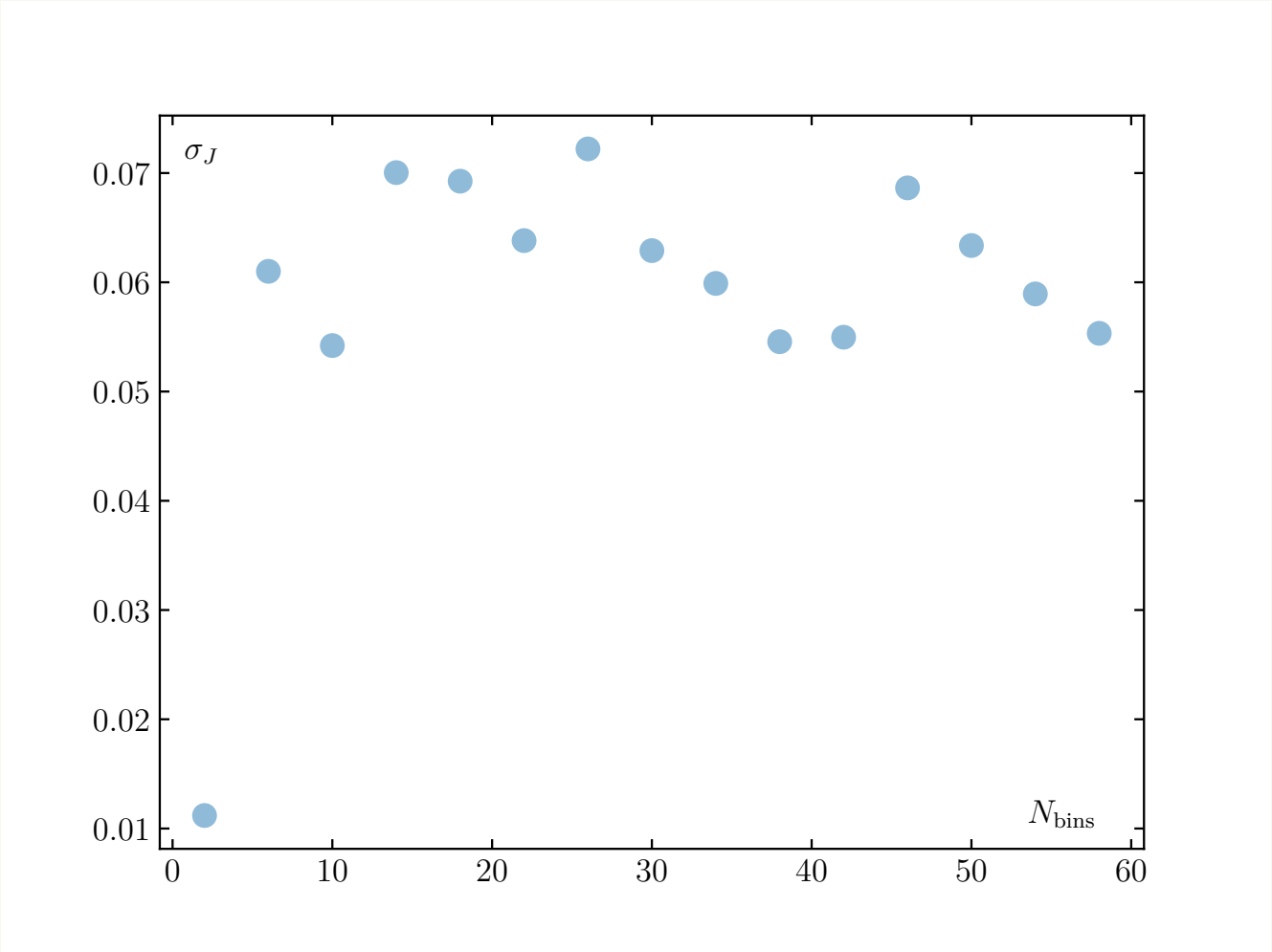
Another example



{ Taken from Wikipedia }

CC5: Verify independence of hyperparameters

Ex: Jackknife bins



Cleaning data

The issue

Eventually you will come across data from many different people/groups.

Possible problems:

- Different conventions
- Different data formats
- Duplicate entries
- Damaged/lost/unlabelled files
- Straight-up wrong numbers

We are in an era where significant effort is put into ensuring data can be **readily used and reproduced**.

🔗 The FAIR guiding princple

Practical goals for your own work

What will save you a lot of pain:

- You know where your data are
- You can read your old data
- Sufficient metadata to know what your data mean
- You can tell if your data got damaged
- Your data are homogeneous and/or well organized
- Your data don't burden supercomputers

Toward these goals

- Bring data into common format
  - Makes it easier to write/debug scripts
  - Helps next person using your work
- Stay consistent with conventions
  - Label files, parameters, output the same way
  - Use the same file type whenever reasonable
  - Follow your group's conventions
- Try to use already-existing data structures (e.g. `yaml` or `csv`)
  - Everyone can leverage already-existing tools
  - Lots of information about format structure

Toward these goals

- Select a format appropriate for the data
  - There is value in data being human-readable
  - Does a 300KB file really need to be a binary?
  - Do you really need to invent a file format from scratch?
  - Parameter files: `yaml`, `xml`, `json`
  - Large data files: `npy` or group's preferred binary
- `tar` together large collections of small files
- Add `README.md` for supplemental information
- **Perform Cross Checks as you go along!**

In general, **don't overcomplicate things!**

Inconsistent conventions

Ex: Unifying different labels

```
# 3 models, 3 different labels for the scaling layer
if MLmodel == 'Bachtis-Aarts-Lucini-Model':
    SL = f'ScaleT{int(scalingLayer)}'
elif MLmodel == 'Fixedparam':
    SL = f'Results_ScaleT{scalingLayer}'
elif MLmodel == 'BBased':
    SL = f'Scale{int(scalingLayer)}'

confs  , Ps = readTable(f'{MLmodel}/{SL}/{T}.d')
confs_E, Es = readTable(f'energy/{T}.d')

for i,conf_M in enumerate(confs_M): # RW needs correct MCMC order
    if conf_M != confs_P[i]:
        logger.TBError(f'MCMC order incorrect for T={T}')
...

# Make SL convention consistent; don't make a new label
writeTable(f'prediction_{MLmodel}_Scale{SL}_T{T}',
           confs, Ps, header=['conf','P'])
```


Responsible bookkeeping

Ex: Where are the data?

```
repo = repository()
repo.append(ensemble(Nt=6, Ns=36, beta=5.85, location='JUWELS',
                     folder='/p/arch2/bipardata/schmidt3',
                     notes='imaginary mu', kind='conf',
                     Nf='2+1', ml=0.00347, ms=0.0938,
                     date='30-06-2025', creator='BiePar'))
...
repo.table()
```

# Nf	ml/ms	Nt	beta	location	folder
2+1	phys	4	5.85	JUWELS	/p/arch2/bipardata/clarke1
2+1	phys	4	5.9	JUWELS	/p/arch2/bipardata/clarke1
2+1	phys	6	5.85	JUWELS	/p/largedata2/bipardata/schmidt3
2+1	phys	6	6.17	JUWELS	/p/largedata2/bipardata/clarke1
2+1	phys	6	6.325	JUWELS	/p/largedata2/bipardata/clarke1
2+1	phys	8	6.135	JUWELS	/p/largedata2/bipardata/schmidt3

Can use 🔗Redmine if you're in the CRC.

Responsible data management

Ex: Archiving data

Aggregating files helps lessen the burden for supercomputers when backing up files, storing their location, etc. Anyway you will have a file limit. Good time to add a **checksum** to verify data integrity.

```
function _archive { # _archive subfolder
  local subfolder=$1
  if [ -d "${subfolder}" ]; then
    tar -cf "${subfolder}.tar" "${subfolder}"
    md5sum "${subfolder}.tar" > "${subfolder}.md5"
  fi
}
function _unarchive { # _unarchive subfolder.tar
  local archive=$1
  if [ -f "${archive}.md5" ]; then md5sum -c "${archive}.md5"; fi
  _checkForFail $? "md5sum"
  tar -xf "${archive}"
}
```


Large tarballs can also be split in chunks and later recombined.

2D Ising model (I)

The 2-d Ising model

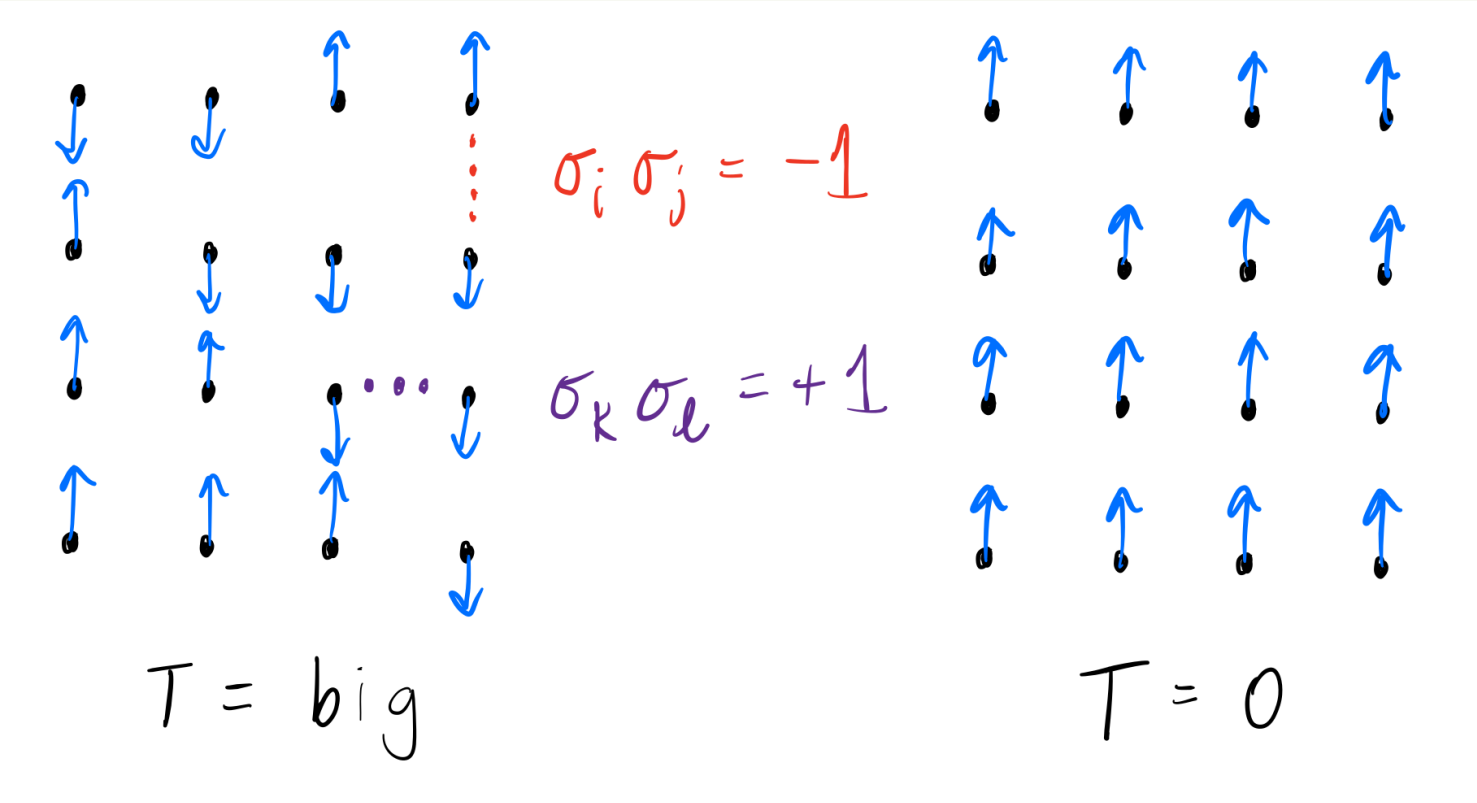
Our context

We are going to practice data cleaning and validation in the context of the 2-d Ising model, a model for a magnet.

- One of the simplest magnet models
- Analytically solvable
- Applications to phase transitions
- Pedagogical for phase transitions

We follow with an introduction/review so that everyone is on the same page.

The 2-d Ising model



$$H = -J \sum_{\langle ij \rangle} \sigma_i \sigma_j \qquad \mathrm{pr}(C) \sim e^{-H(C)/T}$$

Solving Ising with MCMC

Common strategy:

Generate N_{conf} configurations randomly,

↳ then estimate expectation value of observable X

$$\bar{X} = \frac{1}{N_{conf}} \sum_{i=1}^{N_{conf}} X_i$$

Each $X_i = X(C_i)$ measured on C_i from **Markov Chain**

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow \dots$$

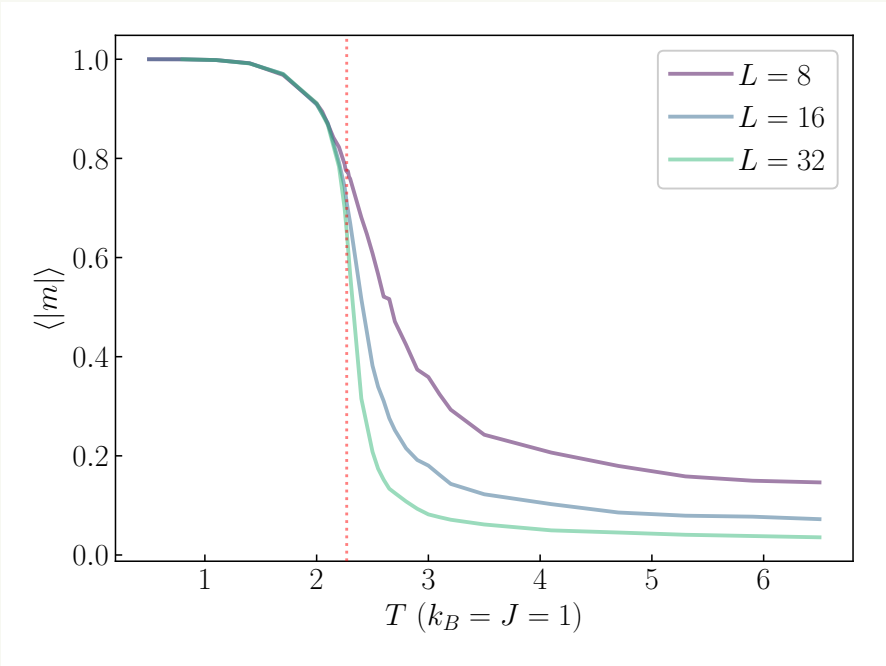
Each C_i drawn from $\mathrm{pr}(C)$

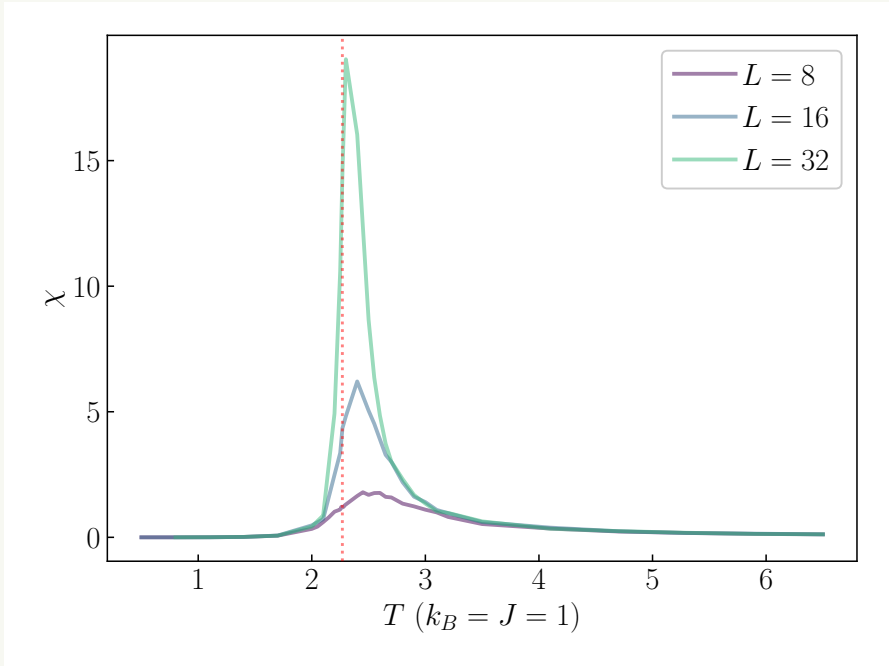
Physics of the Ising model

Order parameter is **magnetization** m

$$m \equiv \frac{1}{V}\sum_i \sigma_i \qquad \chi \equiv \frac{V}{T}\left(\langle |m|^2 \rangle - \langle |m| \rangle^2\right)$$

In $V \to \infty$ limit, m gets a kink and χ diverges at the **critical temperature** T_c.





Your task

Three scientists created 2-d Ising model Markov chains, each managing a different L. They measured energy density and m on each configuration.

1. Develop code to bring all data to common format
2. Write tests for your code! **Examples?**
3. Calculate $\langle|m|\rangle$ and $\langle e\rangle$ $\forall$ T
4. **What kinds of checks for $\langle|m|\rangle$ and $\langle e\rangle$ can you do?**

Eggwin

Beaktrix

Siegfried

Plotting data

Visualizing data for Cross Checks

- Visualizations help understanding
- You will be plotting in your papers
- Gives further opportunities for cross checks!

Ex: Are these data consistent?

```
#            t [ms]        speed [m/s]      accel [m/s^2]
    0.00000000e+00    0.00000000e+00    0.00000000e+00
    1.00000000e+00    5.00000000e-01    1.00000000e+00
    2.00000000e+00    2.00000000e+00    2.00000000e+00
    3.00000000e+00    4.50000000e+00    3.00000000e+00
    4.00000000e+00    8.00000000e+00    4.00000000e+00
    5.00000000e+00    1.25000000e+01    5.00000000e+00
```


Ex: Are these data consistent?

Ex: Are these data consistent?

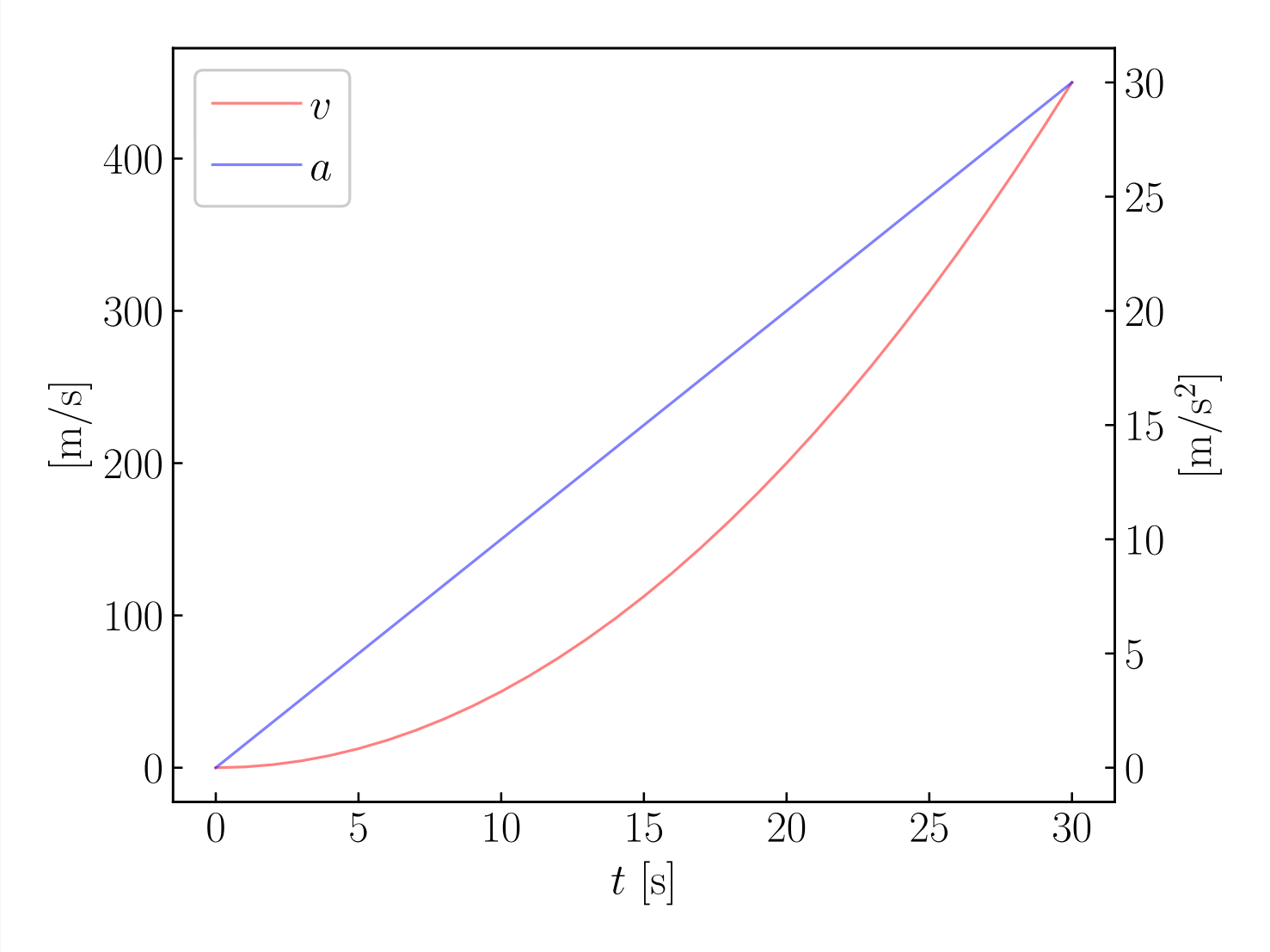
Hard to tell looking at raw data. Much easier to see with plots. In Python, `matplotlib` naturally integrates with your scripts, but use whatever you are comfortable with, e.g. `gnuplot`.

```
# Custom wrappers for common matplotlib tasks
ax1, ax2 = getTwinAxes()
plot_lines(t,speed,marker=None,color='red' ,label='$v$',ax=ax1)
plot_lines(t,accel,marker=None,color='blue',label='$a$',ax=ax2)

# Save people effort decoding your graph by labeling axes
set_params(xlabel='$t$ [s]',ylabel='[m/s]'    ,ax=ax1)
set_params(                 ylabel='[m/s$^2$]',ax=ax2)

plt.show()
```


Ex: Are these data consistent?



Extracting data from a plot

Sometimes need data from a plot that has no corresponding table. Here, I wanted to cross check my HRG energy density against RHS figure.

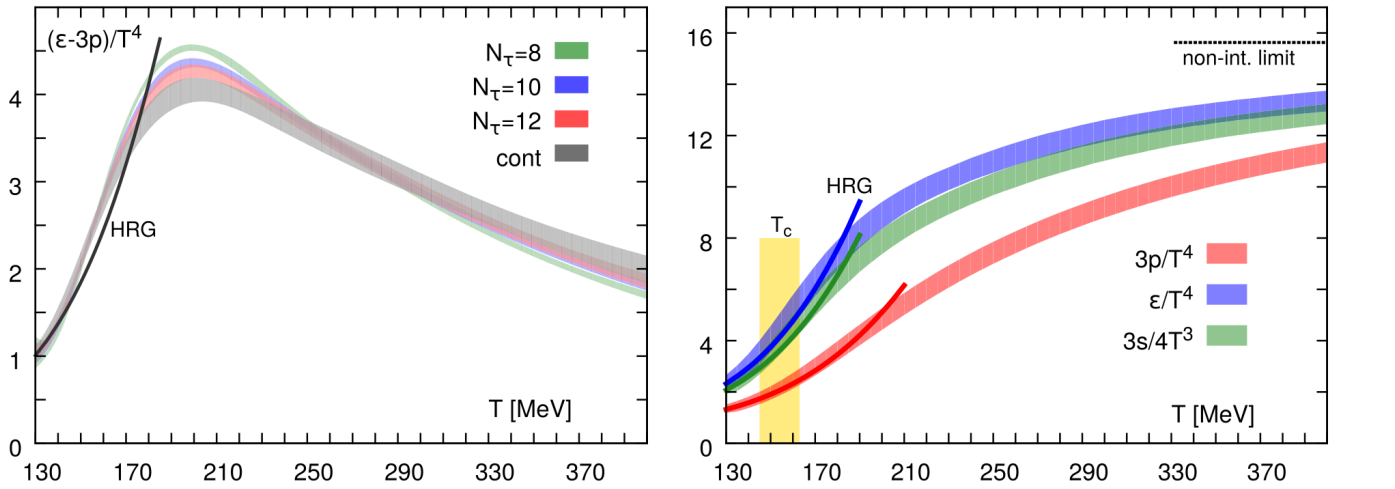
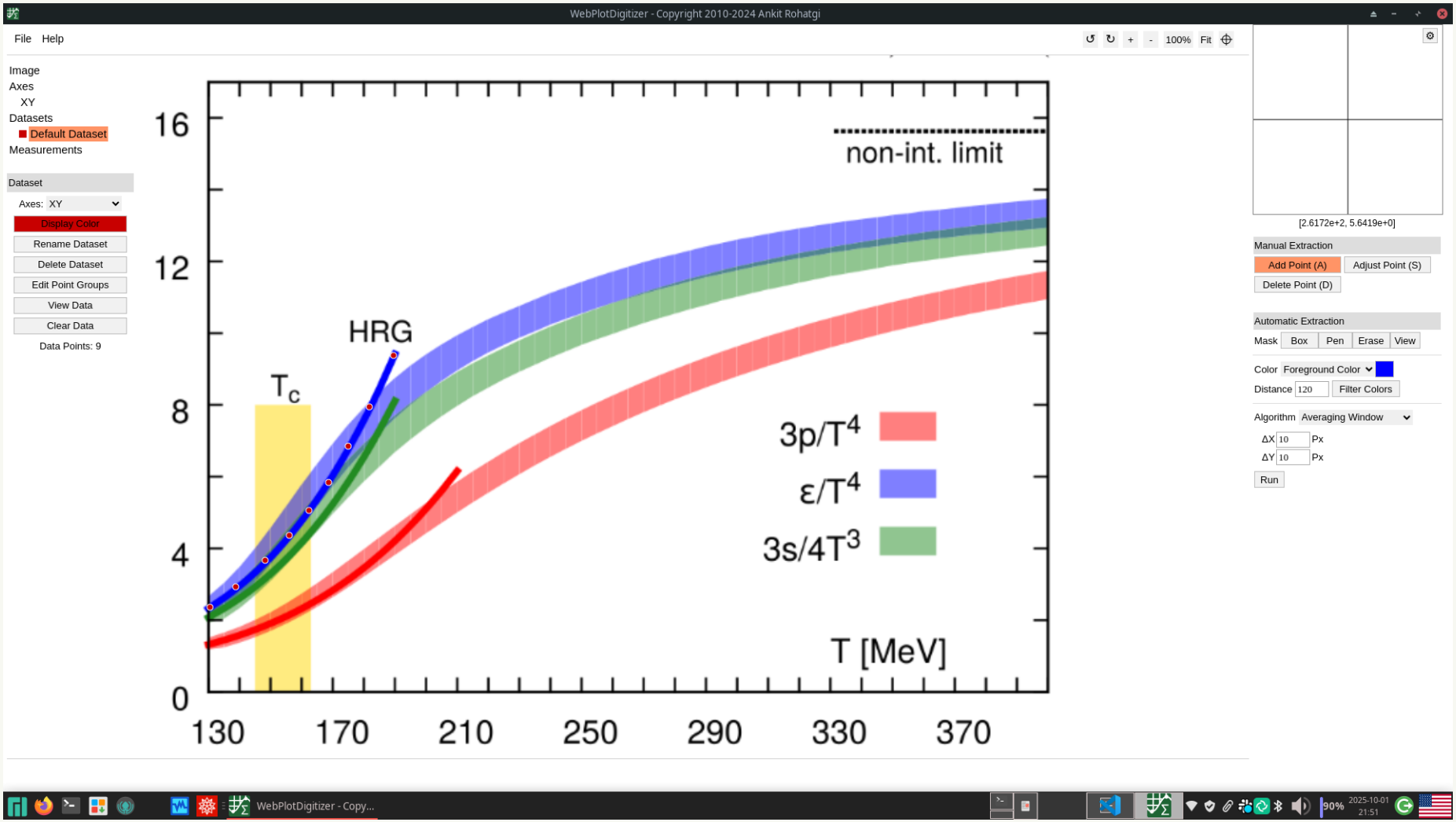


FIG. 5 (color online). Spline fits to the trace anomaly for several values of the lattice spacing $aT = 1/N_\tau$ and the result of our continuum extrapolation (left). Note that the error bands shown here do not include the 2% scale error. The right-hand panel shows suitably normalized pressure, energy density, and entropy density as a function of the temperature. In this case the 2% scale error is included in the error bands. The dark lines show the prediction of the HRG model. The horizontal line at $95\pi^2/60$ in the right panel corresponds to the ideal gas limit for the energy density and the vertical band marks the crossover region, $T_c = (154 \pm 9)$ MeV.

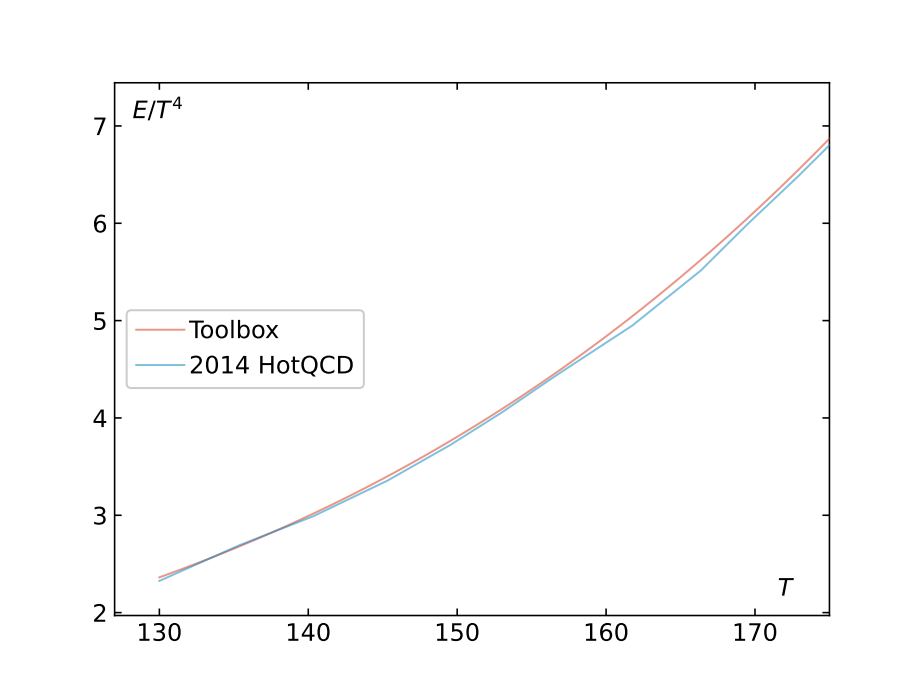
Can use tools like 🔗WebPlotDigitizer

Ex: WebPlotDigitizer



Ex: WebPlotDigitizer

```
#                T [MeV]                    e/T^4
130                       2.325581395348838
135.35885167464113        2.691029900332227
140.33492822966508        2.990033222591364
145.31100478468898        3.355481727574751
...
```



Estimating uncertainties

Reminder: Mean and uncertainty

$C_0 \to C_1 \to C_2 \to \dots$

Mean and variance estimated via

$$\bar{X} = \frac{1}{N_{\text{conf}}} \sum_{i=1}^{N_{\text{conf}}} X_i \qquad \sigma^2 = \frac{1}{N_{\text{conf}} - 1} \sum_{i=1}^{N_{\text{conf}}} \left(X_i^2 - \bar{X}^2 \right)$$

Problem:

C_i is generated as deformation of C_{i-1}. Hence X_i and X_{i-1} are correlated:

$$\sigma_X^2 = \tau_{\text{int}} \frac{\sigma^2}{N_{\text{conf}}} \neq \frac{\sigma^2}{N_{\text{conf}}}$$

with **integrated autocorrelation time** τ_{int}.

Autocorrelation

The relation

$$\sigma_{\mathrm{X}}^2 = \tau_{\mathrm{int}} \frac{\sigma^2}{\mathrm{N_{conf}}}$$

shows there are effectively $\mathrm{N_{conf}}/\tau_{\mathrm{int}}$ independent measurements.

Solutions?

Let $\mathrm{M} \in \mathbb{N}$ s.t. $\mathrm{M} > \tau_{\mathrm{int}}$

- **Decimation**: Keep only measurements X_0, X_{M}, $\mathrm{X}_{2\mathrm{M}}$, etc.
- **Binning**: Construct Y_0 as average over first M measurements, Y_1 as average over the next M, etc.

Tricky uncertainties

Given $\{X_1, \ldots, X_N\}$. Consider some function $f(X)$

- How to get unbiased estimate of $\langle f(X) \rangle$?
- What if f is too complicated for error propagation?

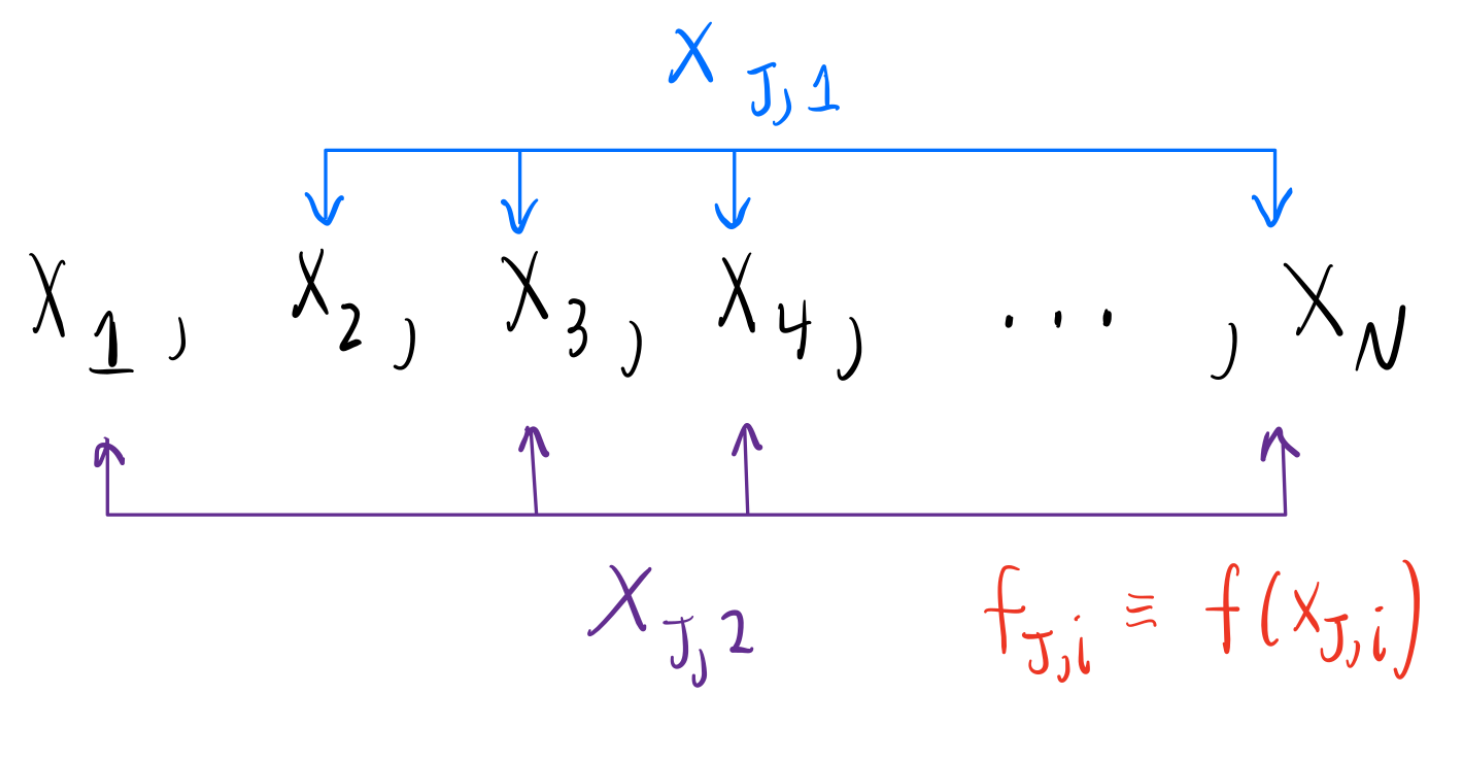
$$\chi \equiv \frac{V}{T} \left(\langle |m|^2 \rangle - \langle |m| \rangle^2 \right)$$

Susceptibility χ is already a variance. How to get its uncertainty?

Can tackle both kinds of problems using **resampling**

Resampling: Jackknife

Create N new samples by dropping a measurement from old sample



Resampling: Jackknife

Jackknifed data are defined by

$$X_{J,i} \equiv \frac{1}{N-1} \sum_{j \neq i} X_j$$

Construct asymptotically unbiased **jackknife estimator** for the mean $\bar{f}_J$:

$$\bar{f}_J \equiv \frac{1}{N} \sum_{i=1}^{N} f_{J,i} \qquad f_{J,i} \equiv f(X_{J,i})$$

The jackknife estimator for the variance of $\bar{f}_J$ is

$$\bar{\sigma}^2_{f_J} = \frac{N-1}{N} \sum_{i=1}^{N} (f_{J,i} - \bar{f}_J)^2$$

Resampling: Jackknife

Ex: Susceptibility

$$\chi \equiv (V/T)\left(\langle |m|^2 \rangle - \langle |m| \rangle^2\right)$$

Can be handled by thinking of $|m|^2$ and $|m|$ as two separate variables x and y. Alternatively can think of susceptibility function f as

$$f(m) \propto \langle |m|^2 \rangle - \langle |m| \rangle^2$$

Either way, the important feature is $|m|^2$ and $|m|$ **are divided into the exact same bins.**

```
def susc(m,V,T):
    return (V/T)*( np.mean(m**2) - np.mean(m)**2 )

chi, chi_err = jackknife(susc, M, args=(V,T), numb_blocks=20)
```

Resampling: Bootstrap

Create N_{boot} new samples by drawing
with replacement from original sample N times

$X_1, X_2, X_3, X_4, \ldots, X_N$

$X_7, X_2, X_2, X_5, \ldots \rightarrow X_{B,1}$

$X_4, X_9, X_1, X_1, \ldots \rightarrow X_{B,2}$

$f_{B,i} \equiv f(X_{B,i})$

Resampling: Bootstrap

Data are bootstrapped **as**

$$X_{B,i} \equiv \frac{1}{N} \sum_{j=1}^{N} n_{ij} X_j$$

where X_j appears n_{ij} times in bin i. As with jackknife, the bootstrap estimator of the mean $\bar{f}_B$ is

$$\bar{f}_B \equiv \frac{1}{N_{\rm boot}} \sum_{i=1}^{N_{\rm boot}} f_{B,i}, \qquad f_{B,i} \equiv f(X_{B,i})$$

The bootstrap estimator for the variance of $\bar{f}_B$ is simply

$$\bar{\sigma}^2_{f_B} = \frac{1}{N_{\rm boot} - 1} \sum_{i=1}^{N_{\rm boot}} (f_{B,i} - \bar{f}_B)^2.$$

Synthesis

Computing some function f of data X_i from a time series

1. Bin the X_i into $X_{B,j}$
2. Resample the $X_{B,j}$ (can be combined with first step)
3. Compute $f(X_{B,j})$ in each resample

Question for you

How can we validate these results?

2D Ising model (II)

Extracting T_c

The **Binder cumulant** is defined as

$$B \equiv 1 - \frac{\langle m^4 \rangle}{3\langle m^2 \rangle^2}$$

- B computed at different L **will cross at** T_c
- $B = 0$ **for** $T \gg T_c$
- $B = 2/3$ **for** $T \ll T_c$
- **Easier to get** T_c **than using** χ

Your task

Now that you've cleaned Eggwin, Beaktrix, and Siegfried's data, we can estimate T_c from our Ising model data. Since we learned about resampling, we can sensibly estimate uncertainties.

1. Develop code to carry out a jackknife (or bootstrap)
2. How can you test this code? Write some tests.
3. Use your code to compute B and its uncertainty
4. What checks are there for B?
5. Plot B vs T and estimate T_c
6. (Optional) Add uncertainties to your $\langle |m| \rangle$ and $\langle e \rangle$ plots
7. (Optional) Plot χ vs T with uncertainties

$$T_c = \frac{2}{\log\left(1+\sqrt{2}\right)}$$